URL Shortener

Engineering Deliverables — AI-Assisted Software Engineering System Assignment

Stack: React + TypeScript · Spring Boot (Java 21) · PostgreSQL | **Status:** Working prototype, verified end-to-end | **Date:** August 5, 2026

This document covers the four deliverables requested from the assignment brief: the architecture overview, the three worked scenarios (greenfield, brownfield, ambiguous), setup instructions, and the testing approach with its limitations and trade-offs. The working prototype itself — source code, tests, and the full High-Level Design — is the companion deliverable this document describes.

1. Architecture overview (components, tools, execution approach, control flow, key decisions)

1.1 Components

A standard layered service on the backend, a thin typed API client on the frontend. The interesting decisions are less about the layering and more about what sits in front of the database on the redirect path (see 1.3, 1.4).

Layer	Component	Responsibility
Frontend	CreateLinkPage, MyLinksPage, LinkAnalyticsPage	React SPA; talks only to the JSON API, never renders a redirect itself
Controller	UrlController	POST/GET/DELETE /api/urls — create, list, deactivate, stats
Controller	RedirectController	GET /r/{shortCode} — the hot path
Service	UrlService, ShortCodeEncoder	alias/URL validation, base62 encoding, expiry logic
Service	ClickTrackingService, AnalyticsService	async click recording; daily/referrer aggregation
Repository	UrlRepository, ClickEventRepository	Spring Data JPA over PostgreSQL
Cache	Caffeine (in-process)	cache-aside for short_code → long_url, TTL + explicit invalidation on delete

1.2 Tools

- **Frontend —** React 18 + TypeScript, Vite, React Router, TanStack React Query for server-state, Vitest + Testing Library for tests, ESLint for static analysis.
- **Backend —** Spring Boot 3 on Java 21, Maven (with Maven Wrapper committed so no local Maven install is required), Spring Data JPA + Hibernate, Flyway for schema migrations, Caffeine for caching, JUnit 5 + Mockito + AssertJ for unit tests, Testcontainers + Awaitility for real-database integration tests.
- **Data —** PostgreSQL 16, schema managed entirely through versioned Flyway migrations (no ddl-auto in any environment).
- **Delivery —** Docker + Docker Compose bring up Postgres, backend, and frontend with a single command; each service also has a documented non-Docker path.

1.3 Execution approach

The system was designed before it was built: a High-Level Design was produced first (component diagram, data model, API surface, sequence flows for create and redirect, and a decisions table) and used as the basis for task decomposition. Implementation then proceeded in dependency order — schema migrations, then entities and repositories, then services, then controllers, then tests, then the frontend — so that each layer had a stable foundation beneath it before being built.

Verification was treated as a first-class task, not an afterthought: rather than trusting that mocked unit tests were sufficient, the actual backend jar was run against a real PostgreSQL instance and the actual frontend was driven through a real headless browser. This is expanded on in Section 4, including two real defects it caught.

1.4 Control flow

Two request paths exist, and they do not share a bottleneck. The creation path goes through the React SPA to the API. The redirect path — a browser following a shortened link out in the wild — goes straight to the API and never touches the SPA at all.

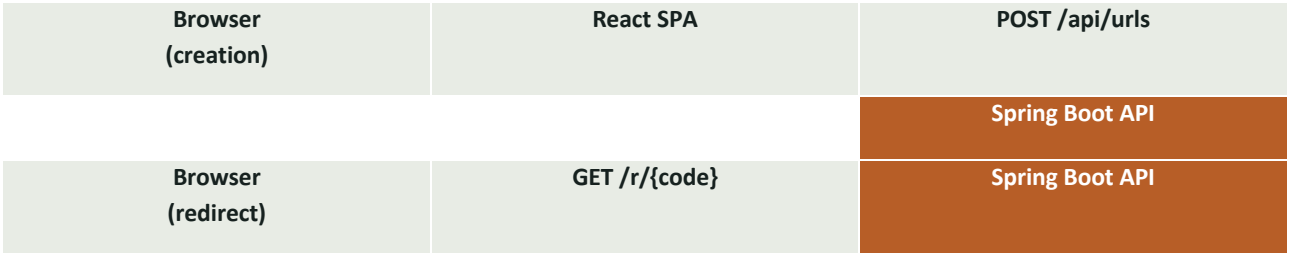

Figure 1 — Creation flows through the SPA; redirect goes directly to the API. Both terminate at the same service, which then reads through Caffeine (cache-aside) before falling back to PostgreSQL.

On the redirect path specifically: the API checks the Caffeine cache first; on a miss it reads PostgreSQL and populates the cache before responding. The 302 is returned immediately — the click event is recorded afterward, off the request thread (@Async), so analytics writes never add latency to a redirect.

1.5 Key decisions

Decision	Alternative considered	Why this way / risk
Base62(sequence id) for short codes	Hash truncation; random + retry; pre-generated key pool	Deterministic, single round trip, zero collisions by construction — no retry logic anywhere. Risk: the sequence is a single-writer bottleneck at extreme scale; mitigation is per-instance id ranges or Snowflake-style ids if this ever runs multi-writer.
/r/{shortCode} namespaced redirects	Bare /{shortCode} at domain root	Keeps redirect routing separable from SPA routes and the rest of the API — a code like api or login can never collide with a real path.
In-process cache (Caffeine)	Redis from day one	Fewer moving parts for a single-instance prototype. Must move to Redis before running more than one API instance, or cache state diverges across nodes.
Async click tracking (@Async, fire-and-forget)	Synchronous insert per redirect	Keeps the redirect response independent of the analytics write path. Trade-off: brief write lag before a click is queryable — acceptable for a dashboard, not for real-time fraud detection.
Soft delete (is_active flag)	Hard delete the row	Preserves click history and enables a genuine 410 Gone vs. 404 Not Found distinction on redirect. Trade-off: the table needs a retention/archival job eventually.
Per-IP rate limiting on POST /api/urls	No abuse control; or full auth	The brief doesn't specify accounts, so rate limiting — not auth — is the abuse control for this iteration.

2. Three scenarios: greenfield, brownfield, ambiguous (each shows decomposition, execution, validation)

2.1 Greenfield — building the system from a blank repository

This covers the entire prototype: nothing existed beforehand.

- **Decomposition —** the raw brief ("core APIs, analytics, and reliability features") was normalized into a High-Level Design first: system context, functional/non-functional requirements, a component table, a data model, an API surface, sequence diagrams for create and redirect, and a decisions table. That document was then broken into an ordered task list — migrations → entities/repositories → services → controllers → tests → frontend — each step depending on the one before it.
- **Execution —** each layer was implemented and compiled/type-checked before moving to the next, rather than writing the whole surface area and debugging it as one block. Backend and frontend were kept independently buildable and testable throughout.
- **Validation —** 20 backend unit/integration tests and 5 frontend tests were written alongside the code they cover, plus a full live run of the actual jar against a real database and the actual UI in a real browser (Section 4) — not just "tests pass," but the golden path clicked through end to end.

2.2 Brownfield — fixing real defects in already-written code

Two genuine brownfield fixes were performed mid-build, both surfaced by running the real system rather than by inspection.

Defect 1 — schema/entity type mismatch

- **Decomposition —** on first real startup against a live PostgreSQL instance, Hibernate's schema validator failed the boot with a column-type mismatch on click_events.country. Root cause: the Flyway migration declared the column CHAR(2), but Hibernate infers VARCHAR(2) for a mapped String field with a length constraint — the two never agreed, and no mocked unit test could have caught it, since none of them touch real DDL.
- **Execution —** fixed at the source of truth (the Flyway migration, changing CHAR(2) to VARCHAR(2)), not papered over with a Hibernate-side override — the schema definition should be the thing that's correct, not worked around.
- **Validation —** the local database was reset, the corrected migration was reapplied from a clean state, and the full application startup plus the API smoke test (create, redirect, conflict, validation, stats) was rerun to confirm the fix, rather than assuming it was fixed.

Defect 2 — browser-only regex failure in the alias field

- **Decomposition —** driving the actual frontend in a real headless Chromium (not jsdom) surfaced a console SyntaxError on the alias input's HTML pattern attribute. Root cause: Chromium's newer unicode-sets ("v"-flag) regex validation mode parses an unescaped trailing dash inside a character class differently than Java's regex engine does — the same-looking pattern was valid on the backend and broken in the browser.

- **Execution —** escaped the dash explicitly ([A-Za-z0-9_\-]{3,16}), which is unambiguous under both parsing modes.
- **Validation —** re-ran the browser-driven script and confirmed zero console errors, then re-captured the create-link screenshot to confirm the field still behaved correctly for a human filling it in.

2.3 Ambiguous requirement — resolving what the brief left open

The brief specifies "core APIs, analytics, and reliability features" but leaves several concrete decisions unstated. Each was resolved explicitly rather than silently assumed.

- **Decomposition —** identified four load-bearing ambiguities before writing code: whether link creation requires authentication; whether custom aliases are supported; whether links expire by default; and how granular analytics should be.
- **Execution —** for each, a concrete choice was made and recorded as an explicit assumption rather than an implicit one: creation is anonymous for this iteration, with a nullable created_by column reserved (not wired up) for a future auth pass so the decision is reversible without a schema redesign; custom aliases are supported, validated against a reserved-word blocklist; links have no default expiry (opt-in only); and analytics is limited to click count, referrer, and coarse geography — deliberately excluding user-level tracking, which also sidesteps PII handling for this scope.
- **Validation —** each assumption is written down in the architecture overview and the Section 4 limitations table rather than left implicit in code, so a reviewer can see exactly what was decided, why, and how narrow the blast radius is if the assumption needs to change later.

3. Setup instructions

3.1 Quick start (Docker Compose)

Requires Docker and Docker Compose only — no local Java, Node, or PostgreSQL install needed.

```
docker compose up --build
```

Service	URL
Frontend	http://localhost:5173
Backend API	http://localhost:8080
Postgres	localhost:5432 (urlshortener / urlshortener)

Open the frontend, paste a URL, shorten it, open My Links to see it, open Stats to watch clicks land. That is the golden path.

3.2 Without Docker

```
# Postgres — bring your own, or just run the db service from compose:
docker compose up db

# Backend (Maven Wrapper included, no local Maven needed)
cd backend
./mvnw spring-boot:run

# Frontend (separate terminal)
cd frontend
npm install
cp .env.example .env   # VITE_API_BASE_URL=http://localhost:8080
npm run dev
```

3.3 Project layout

```
backend/    Spring Boot API (Java 21, Maven)
frontend/   React + TypeScript SPA (Vite)
docker-compose.yml   Postgres + backend + frontend, one command
```

3.4 Configuration

Backend configuration is environment-driven (see backend/src/main/resources/application.yml); the defaults below match docker-compose.

Variable	Default	Purpose
SPRING_DATASOURCE_URL	jdbc:postgresql://localhost:5432/urlshortener	Database connection
APP_BASE_URL	http://localhost:8080	Base URL used to build shortUrl in API responses
APP_CORS_ALLOWED_ORIGIN	http://localhost:5173	Frontend origin allowed to call the API
VITE_API_BASE_URL	http://localhost:8080	Frontend: backend API base URL

4. Testing approach, limitations, and trade-offs

4.1 Backend testing

- **Unit tests —** target the pieces with real logic to get wrong: ShortCodeEncoderTest (determinism, zero collisions across 100,000 ids), LongUrlValidatorTest (scheme allowlist, open-redirect guard), UrlServiceTest (alias conflict, concurrent-alias race, 404-vs-410 branching) — all mocked, no I/O.
- **Integration tests —** UrlControllerIntegrationTest runs the full stack (controller → service → real PostgreSQL via Testcontainers) instead of mocks, because the failure modes that matter here — unique-constraint races, whether an async click write actually lands — only show up against a real engine. Covers create → redirect → stats end to end, duplicate alias, invalid scheme, and the deactivate → 410 (not 404) distinction.
- **Run with:** cd backend && ./mvnw test (spins up a Postgres container automatically; requires Docker).

4.2 Frontend testing

- **StatusPill.test.tsx —** pure logic (active / expired / deactivated branching).
- **CreateLinkPage.test.tsx —** renders through React Query with a mocked fetch, drives the form via Testing Library, asserts both the success path and a server error (409) surfacing through the shared ErrorNotice component.
- **Run with:** cd frontend && npm test, plus npm run lint and npm run build (tsc -b && vite build) as the type-checking/quality gate.

4.3 End-to-end validation actually performed

The development sandbox had no Docker, Maven, or PostgreSQL preinstalled, so validating “it should work” meant provisioning real tooling — a portable JDK 21 and Maven, a standalone PostgreSQL 18 instance, and a headless Chromium via Playwright — and then running the actual backend jar against the actual database, and driving the actual frontend through the real create → list → click → analytics flow. This is what surfaced both defects documented in Section 2.2 before the prototype was called done. It is also the reason both a mocked unit-test layer and a real-dependency integration layer are kept: each catches a class of bug the other structurally cannot.

4.4 Limitations

Area	Limitation	Why it's acceptable for this iteration
Auth	None — creation is anonymous	Not specified in the brief; rate limiting is the abuse control instead
Scale	Single Postgres sequence for id generation; in-process (Caffeine) cache	Fine for one instance; documented upgrade path is per-instance id ranges and Redis if this runs on more than one node
Rate limiting	Keyed on request.getRemoteAddr()	Correct for direct connections; behind a real load balancer this needs X-Forwarded-For instead
Analytics	Click count, referrer, coarse geo only	No user-level tracking — deliberate, sidesteps PII handling for this scope
Custom domains	Not supported	Single shortener domain only
CI/CD	Not included	Out of scope for this exercise; Section 3 covers local and Docker runs

4.5 Trade-offs

The decisions in Section 1.5 each carry a trade-off; the ones most likely to matter to a reviewer are repeated here for visibility:

- Base62 sequence encoding trades a small single-writer bottleneck at extreme scale for zero collisions and zero retry logic everywhere else in the system.
- In-process caching trades multi-instance correctness for operational simplicity in a single-instance prototype — explicitly called out as needing to change before horizontal scaling.
- Async click tracking trades a brief write-lag on analytics for keeping the redirect's critical path completely independent of the analytics write path.
- Soft delete trades unbounded table growth (needs an eventual retention job) for preserved click history and a genuine 410-vs-404 distinction on redirect.